PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char *kw[] = {"int","float","char","double","if","else","while","for","return","void"};
char *ppkw[] = {"include","define","ifdef","ifndef","endif","undef"};

int isKeyword(char s[])
{
   for(int i=0;i<10;i++)
      if(strcmp(s,kw[i])==0)
         return 1;
   return 0;
}

int isPPKeyword(char s[])
{
   for(int i=0;i<6;i++)
      if(strcmp(s,ppkw[i])==0)
         return 1;
   return 0;
}

int main()
{
   FILE *fp = fopen("input.c","r");
   char ch, next, str[100];
   int i;
   if(fp == NULL)
   {
      printf("File not found");
      return 0;
   }

   while((ch = fgetc(fp)) != EOF){
    // Ignore spaces, tabs and newlines
      if(isspace(ch))
         continue;
    // Symbol #
      if(ch == '#'){
         printf("Symbol : #\n");
      }
   // Ignore comments
      else if(ch == '/'){
         next = fgetc(fp);

         if(next == '/')
            while((ch = fgetc(fp)) != '\n' && ch != EOF);
         else if(next == '*'){
            char p = 0;
            while((ch = fgetc(fp)) != EOF){
               if(p == '*' && ch == '/')
                  break;
               p = ch;
            }
```

```
    }
   else{
      printf("Operator : /\n");
      if(next != EOF)
         fseek(fp,-1,SEEK_CUR);
    }
}

// Keywords / Identifiers / Preprocessor Keywords
else if(isalpha(ch) || ch == '_'){
    i = 0;

    do{
       str[i++] = ch;
       ch = fgetc(fp);
    }while(isalnum(ch) || ch == '_');

    str[i] = '\0';

    if(ch != EOF)
       fseek(fp,-1,SEEK_CUR);

    if(isPPKeyword(str))
       printf("Preprocessor Keyword : %s\n", str);
    else if(isKeyword(str))
       printf("Keyword : %s\n", str);
    else
       printf("Identifier : %s\n", str);
}

// Numbers
else if(isdigit(ch)){
    i = 0;

    do{
       str[i++] = ch;
       ch = fgetc(fp);
    }while(isdigit(ch) || ch == '.');

    str[i] = '\0';

    if(ch != EOF)
       fseek(fp,-1,SEEK_CUR);

    printf("Number : %s\n", str);
}

// String Literals
else if(ch == '"'){
    i = 0;

    while((ch = fgetc(fp)) != '"' && ch != EOF)
       str[i++] = ch;

    str[i] = '\0';

    printf("String : \"%s\"\n", str);
```

```
        }

        // Operators
        else if(strchr("+-*=<>!%", ch)){
            next = fgetc(fp);

            if((ch=='='&&next=='=') ||
               (ch=='<'&&next=='=') ||
               (ch=='>'&&next=='=') ||
               (ch=='!'&&next=='=') ||
               (ch=='+'&&next=='+') ||
               (ch=='-'&&next=='-'))

                printf("Operator : %c%c\n", ch, next);

            else{
                printf("Operator : %c\n", ch);

                if(next != EOF)
                    fseek(fp,-1,SEEK_CUR);
            }
        }

        // Special Symbols
        else if(strchr("(){}[];,.:", ch)){
            printf("Symbol : %c\n", ch);
        }

        // Unknown Characters
        else{
            printf("Unknown : %c\n", ch);
        }
    }

    fclose(fp);
    return 0;
}
```

input.c

```
#include <stdio.h>
int main() {
    int a = 10;
    float b = 20.5;

    if(a <= b)
        printf("Hello");

    return 0;
}
```

OUTPUT:

```
dell@dell-OptiPlex-380:~/sandhra$ cc lex.c
dell@dell-OptiPlex-380:~/sandhra$ ./a.out
Symbol : #
Preprocessor Keyword : include
Operator : <
Identifier : stdio
Symbol : .
Identifier : h
Operator : >
Keyword : int
Identifier : main
Symbol : (
Symbol : )
Symbol : {
Keyword : int
Identifier : a
Operator : =
Number : 10
Symbol : ;
Keyword : float
Identifier : b
Operator : =
Number : 20.5
Symbol : ;
Keyword : if
Symbol : (
Identifier : a
Operator : <=
Identifier : b
Symbol : )
Identifier : printf
Symbol : (
String : "Hello"
Symbol : )
Symbol : ;
Keyword : return
Number : 0
Symbol : ;
Symbol : }
dell@dell-OptiPlex-380:~/sandhra$
```

PROGRAM:

```
%{
#include <stdio.h>
%}

%%
int|float|char|if|else|while|return    { printf("%s : Keyword\n", yytext); }
[0-9]+                                 { printf("%s : Number\n", yytext); }
[a-zA-Z_][a-zA-Z0-9_]*                 { printf("%s : Identifier\n", yytext); }
[+\-*/=]                               { printf("%s : Operator\n", yytext); }
[(){};,]                               { printf("%s : Special Symbol\n", yytext); }
[ \t\n]                                ;
.                                      { printf("%s : Unknown\n", yytext); }
%%

int main()
{
   printf("Enter the input:\n");
   yylex();
   return 0;
}

int yywrap()
{
   return 1;
}
```

OUTPUT:

```
dell@dell-OptiPlex-380:~/sandhra$ lex lexical.l
dell@dell-OptiPlex-380:~/sandhra$ gcc lex.yy.c -o lexical -ll
dell@dell-OptiPlex-380:~/sandhra$ ./lexical
Enter the input:
int a=10
int : Keyword
a : Identifier
= : Operator
10 : Number
```

PROGRAM:

yacc.l

```
%{
#include "y.tab.h"
%}

%%

[0-9]+          { return NUMBER; }
[a-zA-Z]+       { return ID; }
[ \t]           ;     /* Ignore spaces */
"+"             { return '+'; }
"-"             { return '-'; }
"*"             { return '*'; }
"/"             { return '/'; }
"("             { return '('; }
")"             { return ')'; }
\n              { return '\n'; }
.               { return yytext[0]; }
%%

int yywrap()
{
   return 1;
}
```

yacc.y

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
void yyerror(char *s);
%}

%token NUMBER ID

%left '+' '-'
%left '*' '/'
%right UMINUS

%%

input:
       expr '\n'       { printf("Valid arithmetic expression\n"); }
       ;

expr:
       expr '+' expr
     | expr '-' expr
     | expr '*' expr
     | expr '/' expr
     | '(' expr ')'
     | '-' expr %prec UMINUS
     | NUMBER
```

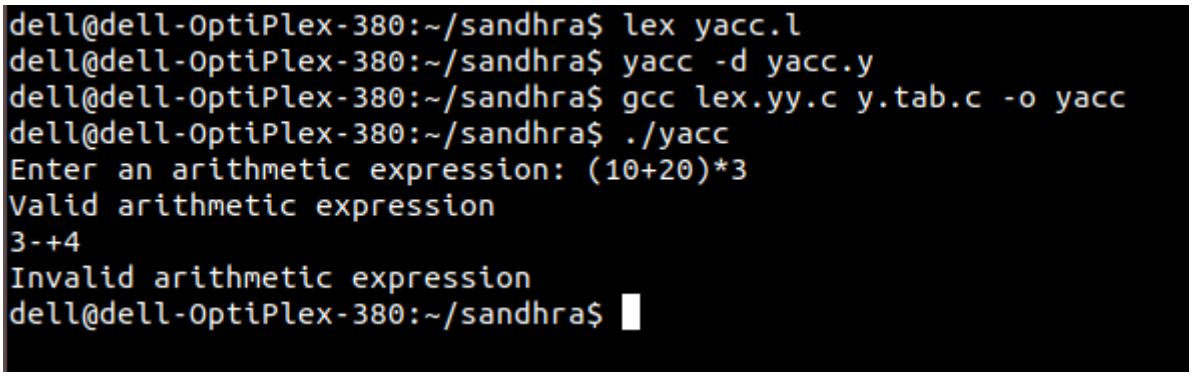
```
    | ID
    ;

%%

void yyerror(char *s)
{
   printf("Invalid arithmetic expression\n");
}

int main()
{
   printf("Enter an arithmetic expression: ");
   yyparse();
   return 0;
}
```

OUTPUT:



PROGRAM:

calc.l

```
%{
#include "y.tab.h"
#include <stdlib.h>
%}

%%
[0-9]+      { yylval = atoi(yytext); return NUMBER; }
[ \t]       ;
\n          { return '\n'; }
.           { return yytext[0]; }
%%

int yywrap()
{
   return 1;
}
```

calc.y

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
void yyerror(char *s);
%}

%token NUMBER

%left '+' '-'
%left '*' '/'

%%

input:
     | input line
     ;

line:
     expr '\n'      { printf("Result = %d\n", $1); }
     ;

expr:
     NUMBER         { $$ = $1; }
   | expr '+' expr  { $$ = $1 + $3; }
   | expr '-' expr  { $$ = $1 - $3; }
   | expr '*' expr  { $$ = $1 * $3; }
   | expr '/' expr
     {
        if($3 == 0)
        {
           printf("Division by zero error\n");
           $$ = 0;
        }
```

```
        else
            $$ = $1 / $3;
    }
   | '(' expr ')'   { $$ = $2; }
   ;

%%

void yyerror(char *s)
{
   printf("Invalid Expression\n");
}

int main()
{
   printf("Enter arithmetic expressions:\n");
   yyparse();
   return 0;
}
```

OUTPUT:

```
dell@dell-OptiPlex-380:~/sandhra$ yacc -d calc.y
dell@dell-OptiPlex-380:~/sandhra$ lex calc.l
dell@dell-OptiPlex-380:~/sandhra$ gcc y.tab.c lex.yy.c -ll
dell@dell-OptiPlex-380:~/sandhra$ ./a.out
Enter arithmetic expressions:
5+3
Result = 8
6*2
Result = 12
10-5
Result = 5
4/2
Result = 2
```

PROGRAM:

```
#include <stdio.h>

int n;
int e[20][20];      // epsilon transition matrix
int visited[20];

void epsilonClosure(int state)
{
   int i;

   visited[state] = 1;
   printf("q%d ", state);

   for(i = 0; i < n; i++)
   {
      if(e[state][i] && !visited[i])
         epsilonClosure(i);
   }
}

int main()
{
   int i, j;

   printf("Enter number of states: ");
   scanf("%d", &n);

   printf("Enter epsilon transition matrix:\n");

   for(i = 0; i < n; i++)
   {
      for(j = 0; j < n; j++)
      {
         scanf("%d", &e[i][j]);
      }
   }

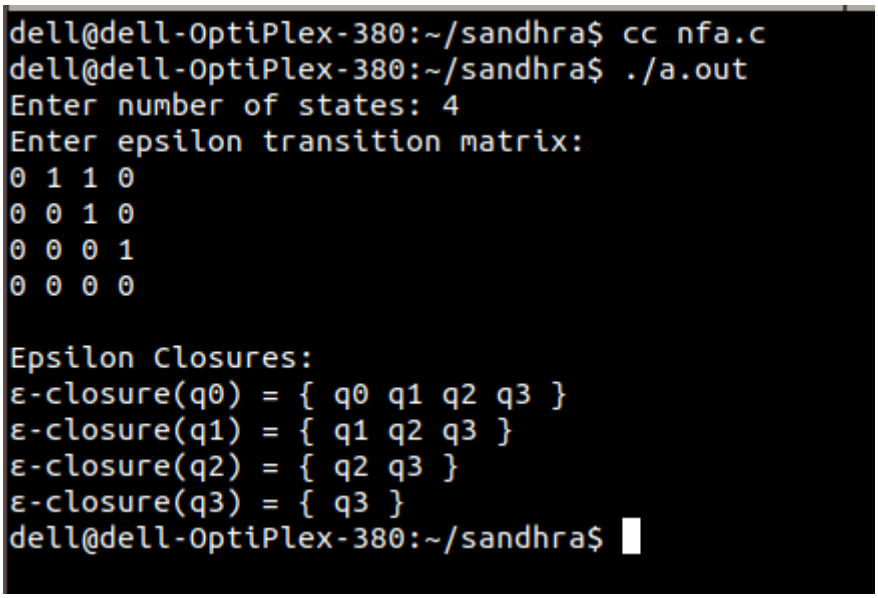
   printf("\nEpsilon Closures:\n");

   for(i = 0; i < n; i++)
   {
      for(j = 0; j < n; j++)
         visited[j] = 0;

      printf("ε-closure(q%d) = { ", i);
      epsilonClosure(i);
      printf("}\n");
   }

   return 0;
}
```

OUTPUT:


dell@dell-OptiPlex-380:~/sandhra$ cc nfa.c
dell@dell-OptiPlex-380:~/sandhra$ ./a.out
Enter number of states: 4
Enter epsilon transition matrix:
0 1 1 0
0 0 1 0
0 0 0 1
0 0 0 0
Epsilon Closures:
ε-closure(q0) = { q0 q1 q2 q3 }
ε-closure(q1) = { q1 q2 q3 }
ε-closure(q2) = { q2 q3 }
ε-closure(q3) = { q3 }
dell@dell-OptiPlex-380:~/sandhra$